

5hwer4myo

October 27, 2025

```
[1]: !pip install scikit-learn pandas joblib gradio shap
```

```
Requirement already satisfied: scikit-learn in
c:\users\priya\anaconda3\lib\site-packages (1.5.1)
Requirement already satisfied: pandas in c:\users\priya\anaconda3\lib\site-
packages (2.2.2)
Requirement already satisfied: joblib in c:\users\priya\anaconda3\lib\site-
packages (1.4.2)
Collecting gradio
  Downloading gradio-5.49.1-py3-none-any.whl.metadata (16 kB)
Collecting shap
  Downloading shap-0.49.1-cp312-cp312-win_amd64.whl.metadata (25 kB)
Requirement already satisfied: numpy>=1.19.5 in
c:\users\priya\anaconda3\lib\site-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in
c:\users\priya\anaconda3\lib\site-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in
c:\users\priya\anaconda3\lib\site-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
c:\users\priya\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
c:\users\priya\anaconda3\lib\site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in
c:\users\priya\anaconda3\lib\site-packages (from pandas) (2023.3)
Collecting aiofiles<25.0,>=22.0 (from gradio)
  Downloading aiofiles-24.1.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: anyio<5.0,>=3.0 in
c:\users\priya\anaconda3\lib\site-packages (from gradio) (4.2.0)
Collecting brotli>=1.1.0 (from gradio)
  Downloading Brotli-1.1.0-cp312-cp312-win_amd64.whl.metadata (5.6 kB)
Collecting fastapi<1.0,>=0.115.2 (from gradio)
  Downloading fastapi-0.119.0-py3-none-any.whl.metadata (28 kB)
Collecting ffmpeg (from gradio)
  Downloading ffmpeg-0.6.3-py3-none-any.whl.metadata (2.9 kB)
Collecting gradio-client==1.13.3 (from gradio)
  Downloading gradio_client-1.13.3-py3-none-any.whl.metadata (7.1 kB)
Collecting groovy~=0.1 (from gradio)
  Downloading groovy-0.1.2-py3-none-any.whl.metadata (6.1 kB)
```

Requirement already satisfied: httpx<1.0,>=0.24.1 in
c:\users\priya\anaconda3\lib\site-packages (from gradio) (0.27.0)
Collecting huggingface-hub<2.0,>=0.33.5 (from gradio)
Downloading huggingface_hub-0.35.3-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: Jinja2<4.0 in c:\users\priya\anaconda3\lib\site-
packages (from gradio) (3.1.4)
Requirement already satisfied: MarkupSafe<4.0,>=2.0 in
c:\users\priya\anaconda3\lib\site-packages (from gradio) (2.1.3)
Collecting orjson~=3.0 (from gradio)
Downloading orjson-3.11.3-cp312-cp312-win_amd64.whl.metadata (43 kB)
Requirement already satisfied: packaging in c:\users\priya\anaconda3\lib\site-
packages (from gradio) (24.1)
Requirement already satisfied: Pillow<12.0,>=8.0 in
c:\users\priya\anaconda3\lib\site-packages (from gradio) (10.4.0)
Requirement already satisfied: Pydantic<2.12,>=2.0 in
c:\users\priya\anaconda3\lib\site-packages (from gradio) (2.8.2)
Collecting pydub (from gradio)
Downloading pydub-0.25.1-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting python-multipart>=0.0.18 (from gradio)
Downloading python_multipart-0.0.20-py3-none-any.whl.metadata (1.8 kB)
Requirement already satisfied: PyYAML<7.0,>=5.0 in
c:\users\priya\anaconda3\lib\site-packages (from gradio) (6.0.1)
Collecting ruff>=0.9.3 (from gradio)
Downloading ruff-0.14.0-py3-none-win_amd64.whl.metadata (26 kB)
Collecting safehttpx<0.2.0,>=0.1.6 (from gradio)
Downloading safehttpx-0.1.6-py3-none-any.whl.metadata (4.2 kB)
Collecting semantic-version~=2.0 (from gradio)
Downloading semantic_version-2.10.0-py2.py3-none-any.whl.metadata (9.7 kB)
Collecting starlette<1.0,>=0.40.0 (from gradio)
Downloading starlette-0.48.0-py3-none-any.whl.metadata (6.3 kB)
Collecting tomlkit<0.14.0,>=0.12.0 (from gradio)
Downloading tomlkit-0.13.3-py3-none-any.whl.metadata (2.8 kB)
Collecting typer<1.0,>=0.12 (from gradio)
Downloading typer-0.19.2-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: typing-extensions~=4.0 in
c:\users\priya\anaconda3\lib\site-packages (from gradio) (4.11.0)
Collecting uvicorn>=0.14.0 (from gradio)
Downloading uvicorn-0.37.0-py3-none-any.whl.metadata (6.6 kB)
Requirement already satisfied: fsspec in c:\users\priya\anaconda3\lib\site-
packages (from gradio-client==1.13.3->gradio) (2024.6.1)
Collecting websockets<16.0,>=13.0 (from gradio-client==1.13.3->gradio)
Downloading websockets-15.0.1-cp312-cp312-win_amd64.whl.metadata (7.0 kB)
Requirement already satisfied: tqdm>=4.27.0 in
c:\users\priya\anaconda3\lib\site-packages (from shap) (4.66.5)
Collecting slicer==0.0.8 (from shap)
Downloading slicer-0.0.8-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: numba>=0.54 in c:\users\priya\anaconda3\lib\site-
packages (from shap) (0.60.0)

Requirement already satisfied: cloudpickle in c:\users\priya\anaconda3\lib\site-packages (from shap) (3.0.0)

Requirement already satisfied: idna>=2.8 in c:\users\priya\anaconda3\lib\site-packages (from anyio<5.0,>=3.0->gradio) (3.7)

Requirement already satisfied: sniffio>=1.1 in c:\users\priya\anaconda3\lib\site-packages (from anyio<5.0,>=3.0->gradio) (1.3.0)

Requirement already satisfied: certifi in c:\users\priya\anaconda3\lib\site-packages (from httpx<1.0,>=0.24.1->gradio) (2025.1.31)

Requirement already satisfied: httpcore==1.* in c:\users\priya\anaconda3\lib\site-packages (from httpx<1.0,>=0.24.1->gradio) (1.0.2)

Requirement already satisfied: h11<0.15,>=0.13 in c:\users\priya\anaconda3\lib\site-packages (from httpcore==1.*->httpx<1.0,>=0.24.1->gradio) (0.14.0)

Requirement already satisfied: filelock in c:\users\priya\anaconda3\lib\site-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (3.13.1)

Requirement already satisfied: requests in c:\users\priya\anaconda3\lib\site-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (2.32.3)

Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in c:\users\priya\anaconda3\lib\site-packages (from numba>=0.54->shap) (0.43.0)

Requirement already satisfied: annotated-types>=0.4.0 in c:\users\priya\anaconda3\lib\site-packages (from pydantic<2.12,>=2.0->gradio) (0.6.0)

Requirement already satisfied: pydantic-core==2.20.1 in c:\users\priya\anaconda3\lib\site-packages (from pydantic<2.12,>=2.0->gradio) (2.20.1)

Requirement already satisfied: six>=1.5 in c:\users\priya\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)

Requirement already satisfied: colorama in c:\users\priya\anaconda3\lib\site-packages (from tqdm>=4.27.0->shap) (0.4.6)

Requirement already satisfied: click>=8.0.0 in c:\users\priya\anaconda3\lib\site-packages (from typer<1.0,>=0.12->gradio) (8.1.7)

Requirement already satisfied: shellingham>=1.3.0 in c:\users\priya\anaconda3\lib\site-packages (from typer<1.0,>=0.12->gradio) (1.5.0)

Requirement already satisfied: rich>=10.11.0 in c:\users\priya\anaconda3\lib\site-packages (from typer<1.0,>=0.12->gradio) (13.7.1)

Requirement already satisfied: markdown-it-py>=2.2.0 in c:\users\priya\anaconda3\lib\site-packages (from rich>=10.11.0->typer<1.0,>=0.12->gradio) (2.2.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\users\priya\anaconda3\lib\site-packages (from rich>=10.11.0->typer<1.0,>=0.12->gradio) (2.15.1)

Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\priya\anaconda3\lib\site-packages (from requests->huggingface-

```

hub<2.0,>=0.33.5->gradio) (3.3.2)
Requirement already satisfied: urllib3<3,>=1.21.1 in
c:\users\priya\anaconda3\lib\site-packages (from requests->huggingface-
hub<2.0,>=0.33.5->gradio) (2.2.3)
Requirement already satisfied: mdurl~=0.1 in c:\users\priya\anaconda3\lib\site-
packages (from markdown-it-py>=2.2.0->rich>=10.11.0->typer<1.0,>=0.12->gradio)
(0.1.0)
Downloading gradio-5.49.1-py3-none-any.whl (63.5 MB)

```

```

----- 0.0/63.5 MB ? eta -:--:--
----- 0.3/63.5 MB ? eta -:--:--
----- 0.5/63.5 MB 1.4 MB/s eta 0:00:46
----- 0.8/63.5 MB 1.4 MB/s eta 0:00:45
----- 1.0/63.5 MB 1.2 MB/s eta 0:00:53
----- 1.3/63.5 MB 1.2 MB/s eta 0:00:51
----- 1.6/63.5 MB 1.3 MB/s eta 0:00:47
- ----- 2.1/63.5 MB 1.3 MB/s eta 0:00:46
- ----- 2.4/63.5 MB 1.3 MB/s eta 0:00:46
- ----- 2.6/63.5 MB 1.4 MB/s eta 0:00:44
- ----- 3.1/63.5 MB 1.5 MB/s eta 0:00:41
-- ----- 3.4/63.5 MB 1.5 MB/s eta 0:00:41
-- ----- 3.7/63.5 MB 1.4 MB/s eta 0:00:43
-- ----- 3.9/63.5 MB 1.4 MB/s eta 0:00:43
-- ----- 4.2/63.5 MB 1.4 MB/s eta 0:00:43
-- ----- 4.2/63.5 MB 1.4 MB/s eta 0:00:43
-- ----- 4.5/63.5 MB 1.3 MB/s eta 0:00:44
-- ----- 4.7/63.5 MB 1.3 MB/s eta 0:00:44
--- ----- 5.0/63.5 MB 1.3 MB/s eta 0:00:45
--- ----- 5.5/63.5 MB 1.4 MB/s eta 0:00:43
--- ----- 5.8/63.5 MB 1.4 MB/s eta 0:00:43
--- ----- 6.0/63.5 MB 1.4 MB/s eta 0:00:42
--- ----- 6.6/63.5 MB 1.4 MB/s eta 0:00:42
--- ----- 6.8/63.5 MB 1.4 MB/s eta 0:00:41
--- ----- 7.3/63.5 MB 1.4 MB/s eta 0:00:40
--- ----- 7.9/63.5 MB 1.5 MB/s eta 0:00:38
--- ----- 8.1/63.5 MB 1.5 MB/s eta 0:00:38
--- ----- 8.7/63.5 MB 1.5 MB/s eta 0:00:37
--- ----- 8.9/63.5 MB 1.5 MB/s eta 0:00:36
--- ----- 9.2/63.5 MB 1.5 MB/s eta 0:00:36
--- ----- 9.7/63.5 MB 1.5 MB/s eta 0:00:36
--- ----- 10.0/63.5 MB 1.5 MB/s eta 0:00:35
--- ----- 10.2/63.5 MB 1.5 MB/s eta 0:00:35
--- ----- 10.7/63.5 MB 1.5 MB/s eta 0:00:35
--- ----- 11.0/63.5 MB 1.5 MB/s eta 0:00:35
--- ----- 11.5/63.5 MB 1.6 MB/s eta 0:00:34
--- ----- 12.1/63.5 MB 1.6 MB/s eta 0:00:33
--- ----- 12.8/63.5 MB 1.6 MB/s eta 0:00:32
--- ----- 13.4/63.5 MB 1.7 MB/s eta 0:00:31
--- ----- 14.2/63.5 MB 1.7 MB/s eta 0:00:30

```

```

----- 14.7/63.5 MB 1.7 MB/s eta 0:00:29
----- 15.2/63.5 MB 1.8 MB/s eta 0:00:28
----- 15.7/63.5 MB 1.8 MB/s eta 0:00:28
----- 16.5/63.5 MB 1.8 MB/s eta 0:00:27
----- 17.0/63.5 MB 1.8 MB/s eta 0:00:26
----- 17.8/63.5 MB 1.9 MB/s eta 0:00:25
----- 18.4/63.5 MB 1.9 MB/s eta 0:00:24
----- 19.1/63.5 MB 1.9 MB/s eta 0:00:24
----- 19.9/63.5 MB 2.0 MB/s eta 0:00:23
----- 20.4/63.5 MB 2.0 MB/s eta 0:00:22
----- 21.5/63.5 MB 2.0 MB/s eta 0:00:21
----- 21.8/63.5 MB 2.0 MB/s eta 0:00:21
----- 22.3/63.5 MB 2.0 MB/s eta 0:00:21
----- 22.8/63.5 MB 2.0 MB/s eta 0:00:20
----- 23.6/63.5 MB 2.1 MB/s eta 0:00:20
----- 24.4/63.5 MB 2.1 MB/s eta 0:00:19
----- 24.9/63.5 MB 2.1 MB/s eta 0:00:19
----- 25.7/63.5 MB 2.1 MB/s eta 0:00:18
----- 26.5/63.5 MB 2.1 MB/s eta 0:00:18
----- 26.7/63.5 MB 2.2 MB/s eta 0:00:18
----- 27.0/63.5 MB 2.1 MB/s eta 0:00:18
----- 27.5/63.5 MB 2.1 MB/s eta 0:00:17
----- 28.0/63.5 MB 2.1 MB/s eta 0:00:17
----- 28.8/63.5 MB 2.2 MB/s eta 0:00:17
----- 29.6/63.5 MB 2.2 MB/s eta 0:00:16
----- 30.1/63.5 MB 2.2 MB/s eta 0:00:16
----- 30.9/63.5 MB 2.2 MB/s eta 0:00:15
----- 31.5/63.5 MB 2.2 MB/s eta 0:00:15
----- 32.2/63.5 MB 2.2 MB/s eta 0:00:15
----- 32.8/63.5 MB 2.2 MB/s eta 0:00:14
----- 33.6/63.5 MB 2.3 MB/s eta 0:00:14
----- 34.6/63.5 MB 2.3 MB/s eta 0:00:13
----- 34.6/63.5 MB 2.3 MB/s eta 0:00:13
----- 34.9/63.5 MB 2.3 MB/s eta 0:00:13
----- 35.4/63.5 MB 2.3 MB/s eta 0:00:13
----- 36.2/63.5 MB 2.3 MB/s eta 0:00:13
----- 36.7/63.5 MB 2.3 MB/s eta 0:00:12
----- 37.0/63.5 MB 2.3 MB/s eta 0:00:12
----- 37.0/63.5 MB 2.3 MB/s eta 0:00:12
----- 38.0/63.5 MB 2.3 MB/s eta 0:00:12
----- 38.5/63.5 MB 2.3 MB/s eta 0:00:12
----- 39.3/63.5 MB 2.3 MB/s eta 0:00:11
----- 40.1/63.5 MB 2.3 MB/s eta 0:00:11
----- 40.6/63.5 MB 2.3 MB/s eta 0:00:10
----- 41.2/63.5 MB 2.3 MB/s eta 0:00:10
----- 41.9/63.5 MB 2.3 MB/s eta 0:00:10
----- 42.5/63.5 MB 2.3 MB/s eta 0:00:10
----- 43.3/63.5 MB 2.3 MB/s eta 0:00:09

```

```

----- 44.0/63.5 MB 2.4 MB/s eta 0:00:09
----- 44.8/63.5 MB 2.4 MB/s eta 0:00:08
----- 45.6/63.5 MB 2.4 MB/s eta 0:00:08
----- 46.7/63.5 MB 2.4 MB/s eta 0:00:07
----- 47.2/63.5 MB 2.4 MB/s eta 0:00:07
----- 47.7/63.5 MB 2.4 MB/s eta 0:00:07
----- 48.5/63.5 MB 2.4 MB/s eta 0:00:07
----- 49.3/63.5 MB 2.4 MB/s eta 0:00:06
----- 49.5/63.5 MB 2.5 MB/s eta 0:00:06
----- 50.1/63.5 MB 2.4 MB/s eta 0:00:06
----- 50.6/63.5 MB 2.4 MB/s eta 0:00:06
----- 50.9/63.5 MB 2.4 MB/s eta 0:00:06
----- 51.4/63.5 MB 2.4 MB/s eta 0:00:06
----- 51.6/63.5 MB 2.4 MB/s eta 0:00:05
----- 52.4/63.5 MB 2.4 MB/s eta 0:00:05
----- 53.2/63.5 MB 2.4 MB/s eta 0:00:05
----- 53.7/63.5 MB 2.4 MB/s eta 0:00:05
----- 54.3/63.5 MB 2.4 MB/s eta 0:00:04
----- 54.8/63.5 MB 2.4 MB/s eta 0:00:04
----- 55.3/63.5 MB 2.4 MB/s eta 0:00:04
----- 55.6/63.5 MB 2.4 MB/s eta 0:00:04
----- 56.1/63.5 MB 2.4 MB/s eta 0:00:04
----- 56.4/63.5 MB 2.4 MB/s eta 0:00:03
----- 56.9/63.5 MB 2.4 MB/s eta 0:00:03
----- 57.1/63.5 MB 2.4 MB/s eta 0:00:03
----- 57.7/63.5 MB 2.4 MB/s eta 0:00:03
----- 58.2/63.5 MB 2.4 MB/s eta 0:00:03
----- 58.7/63.5 MB 2.4 MB/s eta 0:00:02
----- 59.2/63.5 MB 2.4 MB/s eta 0:00:02
----- 59.5/63.5 MB 2.4 MB/s eta 0:00:02
----- 60.0/63.5 MB 2.4 MB/s eta 0:00:02
----- 60.8/63.5 MB 2.4 MB/s eta 0:00:02
----- 61.3/63.5 MB 2.4 MB/s eta 0:00:01
----- 61.9/63.5 MB 2.4 MB/s eta 0:00:01
----- 62.7/63.5 MB 2.4 MB/s eta 0:00:01
----- 63.4/63.5 MB 2.4 MB/s eta 0:00:01
----- 63.4/63.5 MB 2.4 MB/s eta 0:00:01
----- 63.5/63.5 MB 2.4 MB/s eta 0:00:00
Downloading gradio_client-1.13.3-py3-none-any.whl (325 kB)
Downloading shap-0.49.1-cp312-cp312-win_amd64.whl (548 kB)
----- 0.0/548.0 kB ? eta -:--:--
----- 524.3/548.0 kB 3.4 MB/s eta 0:00:01
----- 548.0/548.0 kB 1.8 MB/s eta 0:00:00
Downloading slicer-0.0.8-py3-none-any.whl (15 kB)
Downloading aiofiles-24.1.0-py3-none-any.whl (15 kB)
Downloading Brotli-1.1.0-cp312-cp312-win_amd64.whl (357 kB)
Downloading fastapi-0.119.0-py3-none-any.whl (107 kB)
Downloading groovy-0.1.2-py3-none-any.whl (14 kB)

```

```

Downloading huggingface_hub-0.35.3-py3-none-any.whl (564 kB)
----- 0.0/564.3 kB ? eta -:-:--
----- -- 524.3/564.3 kB 3.4 MB/s eta 0:00:01
----- 564.3/564.3 kB 1.9 MB/s eta 0:00:00
Downloading orjson-3.11.3-cp312-cp312-win_amd64.whl (131 kB)
Downloading python_multipart-0.0.20-py3-none-any.whl (24 kB)
Downloading ruff-0.14.0-py3-none-win_amd64.whl (13.5 MB)
----- 0.0/13.5 MB ? eta -:-:--
- ----- 0.5/13.5 MB 4.2 MB/s eta 0:00:04
--- ----- 1.0/13.5 MB 3.9 MB/s eta 0:00:04
--- ----- 1.3/13.5 MB 2.4 MB/s eta 0:00:06
----- 1.6/13.5 MB 2.0 MB/s eta 0:00:06
----- 1.6/13.5 MB 2.0 MB/s eta 0:00:06
----- 1.6/13.5 MB 2.0 MB/s eta 0:00:06
----- 1.8/13.5 MB 1.2 MB/s eta 0:00:10
----- 2.1/13.5 MB 1.2 MB/s eta 0:00:10
----- 2.6/13.5 MB 1.3 MB/s eta 0:00:09
----- 3.1/13.5 MB 1.4 MB/s eta 0:00:08
----- 3.4/13.5 MB 1.5 MB/s eta 0:00:07
----- 3.9/13.5 MB 1.5 MB/s eta 0:00:07
----- 4.2/13.5 MB 1.5 MB/s eta 0:00:07
----- 4.7/13.5 MB 1.5 MB/s eta 0:00:06
----- 5.2/13.5 MB 1.6 MB/s eta 0:00:06
----- 6.0/13.5 MB 1.7 MB/s eta 0:00:05
----- 6.8/13.5 MB 1.8 MB/s eta 0:00:04
----- 7.3/13.5 MB 1.9 MB/s eta 0:00:04
----- 7.9/13.5 MB 1.9 MB/s eta 0:00:03
----- 8.4/13.5 MB 1.9 MB/s eta 0:00:03
----- 8.7/13.5 MB 1.9 MB/s eta 0:00:03
----- 8.9/13.5 MB 1.9 MB/s eta 0:00:03
----- 9.2/13.5 MB 1.8 MB/s eta 0:00:03
----- 9.4/13.5 MB 1.8 MB/s eta 0:00:03
----- 9.4/13.5 MB 1.8 MB/s eta 0:00:03
----- 9.7/13.5 MB 1.8 MB/s eta 0:00:03
----- 10.0/13.5 MB 1.7 MB/s eta 0:00:03
----- 10.2/13.5 MB 1.7 MB/s eta 0:00:02
----- 10.5/13.5 MB 1.7 MB/s eta 0:00:02
----- 11.0/13.5 MB 1.7 MB/s eta 0:00:02
----- 11.5/13.5 MB 1.7 MB/s eta 0:00:02
----- 12.1/13.5 MB 1.8 MB/s eta 0:00:01
----- -- 12.6/13.5 MB 1.8 MB/s eta 0:00:01
----- - 12.8/13.5 MB 1.8 MB/s eta 0:00:01
----- - 12.8/13.5 MB 1.8 MB/s eta 0:00:01
----- 13.4/13.5 MB 1.7 MB/s eta 0:00:01
----- 13.5/13.5 MB 1.7 MB/s eta 0:00:00
Downloading safehttpx-0.1.6-py3-none-any.whl (8.7 kB)
Downloading semantic_version-2.10.0-py2.py3-none-any.whl (15 kB)
Downloading starlette-0.48.0-py3-none-any.whl (73 kB)

```

```

Downloading tomlkit-0.13.3-py3-none-any.whl (38 kB)
Downloading typer-0.19.2-py3-none-any.whl (46 kB)
Downloading uvicorn-0.37.0-py3-none-any.whl (67 kB)
Downloading ffmpeg-0.6.3-py3-none-any.whl (5.5 kB)
Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)
Downloading websockets-15.0.1-cp312-cp312-win_amd64.whl (176 kB)
Installing collected packages: pydub, brotli, websockets, tomlkit, slicer,
semantic-version, ruff, python-multipart, orjson, groovy, ffmpeg, aiofiles,
uvicorn, starlette, huggingface-hub, typer, shap, safehttpx, gradio-client,
fastapi, gradio
  Attempting uninstall: brotli
    Found existing installation: Brotli 1.0.9
    Uninstalling Brotli-1.0.9:
      Successfully uninstalled Brotli-1.0.9
  Attempting uninstall: tomlkit
    Found existing installation: tomlkit 0.11.1
    Uninstalling tomlkit-0.11.1:
      Successfully uninstalled tomlkit-0.11.1
  Attempting uninstall: typer
    Found existing installation: typer 0.9.0
    Uninstalling typer-0.9.0:
      Successfully uninstalled typer-0.9.0
Successfully installed aiofiles-24.1.0 brotli-1.1.0 fastapi-0.119.0 ffmpeg-0.6.3
gradio-5.49.1 gradio-client-1.13.3 groovy-0.1.2 huggingface-hub-0.35.3
orjson-3.11.3 pydub-0.25.1 python-multipart-0.0.20 ruff-0.14.0 safehttpx-0.1.6
semantic-version-2.10.0 shap-0.49.1 slicer-0.0.8 starlette-0.48.0 tomlkit-0.13.3
typer-0.19.2 uvicorn-0.37.0 websockets-15.0.1

```

```
[5]: import pandas as pd
```

```

data = pd.read_csv("ai4i2020.csv")
data.head()
print(data.columns.tolist())

```

```

['UDI', 'Product ID', 'Type', 'Air_temperature', 'Process_temperature ',
'Rotational_speed ', 'Torque ', 'Tool_wear', 'Machine_failure', 'TWF', 'HDF',
'PWF', 'OSF', 'RNF']

```

```

[9]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Example columns (adjust based on the CSV)
# Clean column names to remove leading/trailing spaces
data.columns = data.columns.str.strip()

# Example columns (adjusted based on cleaned CSV)
FEATURES = ['Air_temperature', 'Process_temperature',

```



```

        'Rotational_speed', 'Torque',
        'Tool_wear']
TARGET = 'Machine_failure' # adjust column name if needed

# Features & target
X = data[FEATURES]
y = data[TARGET]

# Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

[11]: from sklearn.ensemble import RandomForestClassifier
      from sklearn.metrics import classification_report, confusion_matrix
      import joblib

      # Train RandomForest
      model = RandomForestClassifier(n_estimators=100, random_state=42)
      model.fit(X_train_scaled, y_train)

      # Evaluate
      y_pred = model.predict(X_test_scaled)
      print("Classification Report:\n", classification_report(y_test, y_pred))
      print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

      # Save model & scaler
      joblib.dump(model, "model_ai4i_rf.joblib")
      joblib.dump(scaler, "scaler_ai4i.joblib")

```

Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	1932
1	0.90	0.63	0.74	68
accuracy			0.98	2000
macro avg	0.94	0.81	0.87	2000
weighted avg	0.98	0.98	0.98	2000

Confusion Matrix:

```
[[1927  5]
```

```
[ 25  43]]
```

```
[11]: ['scaler_ai4i.joblib']
```

```
[15]: !pip install xgboost
```

Collecting xgboost

Downloading xgboost-3.0.5-py3-none-win_amd64.whl.metadata (2.1 kB)

Requirement already satisfied: numpy in c:\users\priya\anaconda3\lib\site-packages (from xgboost) (1.26.4)

Requirement already satisfied: scipy in c:\users\priya\anaconda3\lib\site-packages (from xgboost) (1.13.1)

Downloading xgboost-3.0.5-py3-none-win_amd64.whl (56.8 MB)

```
----- 0.0/56.8 MB ? eta -:--:--
----- 0.3/56.8 MB ? eta -:--:--
----- 0.8/56.8 MB 2.4 MB/s eta 0:00:24
- ----- 1.6/56.8 MB 2.7 MB/s eta 0:00:21
- ----- 2.1/56.8 MB 2.9 MB/s eta 0:00:20
- ----- 2.6/56.8 MB 2.8 MB/s eta 0:00:20
-- ----- 2.9/56.8 MB 2.8 MB/s eta 0:00:20
-- ----- 3.7/56.8 MB 2.5 MB/s eta 0:00:21
-- ----- 4.2/56.8 MB 2.5 MB/s eta 0:00:21
--- ----- 4.7/56.8 MB 2.5 MB/s eta 0:00:21
--- ----- 5.0/56.8 MB 2.4 MB/s eta 0:00:22
--- ----- 5.2/56.8 MB 2.4 MB/s eta 0:00:22
---- ----- 5.8/56.8 MB 2.4 MB/s eta 0:00:22
---- ----- 6.3/56.8 MB 2.4 MB/s eta 0:00:22
---- ----- 6.8/56.8 MB 2.3 MB/s eta 0:00:22
---- ----- 6.8/56.8 MB 2.3 MB/s eta 0:00:22
----- ----- 7.3/56.8 MB 2.2 MB/s eta 0:00:23
----- ----- 7.6/56.8 MB 2.2 MB/s eta 0:00:23
----- ----- 7.9/56.8 MB 2.1 MB/s eta 0:00:24
----- ----- 8.4/56.8 MB 2.1 MB/s eta 0:00:23
----- ----- 8.7/56.8 MB 2.1 MB/s eta 0:00:23
----- ----- 9.4/56.8 MB 2.1 MB/s eta 0:00:23
----- ----- 9.7/56.8 MB 2.2 MB/s eta 0:00:22
----- ----- 10.2/56.8 MB 2.1 MB/s eta 0:00:22
----- ----- 10.7/56.8 MB 2.1 MB/s eta 0:00:22
----- ----- 11.3/56.8 MB 2.1 MB/s eta 0:00:22
----- ----- 12.1/56.8 MB 2.2 MB/s eta 0:00:21
----- ----- 12.3/56.8 MB 2.2 MB/s eta 0:00:21
----- ----- 12.8/56.8 MB 2.2 MB/s eta 0:00:21
----- ----- 13.4/56.8 MB 2.2 MB/s eta 0:00:20
----- ----- 13.6/56.8 MB 2.2 MB/s eta 0:00:20
----- ----- 13.6/56.8 MB 2.2 MB/s eta 0:00:20
----- ----- 14.2/56.8 MB 2.1 MB/s eta 0:00:21
----- ----- 14.4/56.8 MB 2.1 MB/s eta 0:00:21
----- ----- 14.7/56.8 MB 2.1 MB/s eta 0:00:21
```

```

----- 14.9/56.8 MB 2.0 MB/s eta 0:00:21
----- 15.2/56.8 MB 2.0 MB/s eta 0:00:21
----- 15.7/56.8 MB 2.0 MB/s eta 0:00:21
----- 16.3/56.8 MB 2.0 MB/s eta 0:00:21
----- 16.5/56.8 MB 2.0 MB/s eta 0:00:20
----- 16.8/56.8 MB 2.0 MB/s eta 0:00:21
----- 17.3/56.8 MB 2.0 MB/s eta 0:00:20
----- 17.3/56.8 MB 2.0 MB/s eta 0:00:20
----- 17.3/56.8 MB 2.0 MB/s eta 0:00:20
----- 17.6/56.8 MB 1.9 MB/s eta 0:00:21
----- 18.1/56.8 MB 1.9 MB/s eta 0:00:21
----- 18.6/56.8 MB 1.9 MB/s eta 0:00:20
----- 18.9/56.8 MB 1.9 MB/s eta 0:00:20
----- 19.4/56.8 MB 1.9 MB/s eta 0:00:20
----- 19.9/56.8 MB 1.9 MB/s eta 0:00:20
----- 20.7/56.8 MB 2.0 MB/s eta 0:00:19
----- 21.2/56.8 MB 2.0 MB/s eta 0:00:19
----- 21.8/56.8 MB 2.0 MB/s eta 0:00:18
----- 22.3/56.8 MB 2.0 MB/s eta 0:00:18
----- 23.1/56.8 MB 2.0 MB/s eta 0:00:17
----- 23.3/56.8 MB 2.0 MB/s eta 0:00:17
----- 23.6/56.8 MB 2.0 MB/s eta 0:00:17
----- 23.9/56.8 MB 2.0 MB/s eta 0:00:17
----- 24.4/56.8 MB 2.0 MB/s eta 0:00:17
----- 24.9/56.8 MB 2.0 MB/s eta 0:00:16
----- 25.4/56.8 MB 2.0 MB/s eta 0:00:16
----- 26.0/56.8 MB 2.0 MB/s eta 0:00:16
----- 26.2/56.8 MB 2.0 MB/s eta 0:00:16
----- 26.7/56.8 MB 2.0 MB/s eta 0:00:15
----- 27.3/56.8 MB 2.0 MB/s eta 0:00:15
----- 27.5/56.8 MB 2.0 MB/s eta 0:00:15
----- 28.0/56.8 MB 2.0 MB/s eta 0:00:15
----- 28.6/56.8 MB 2.0 MB/s eta 0:00:14
----- 29.1/56.8 MB 2.0 MB/s eta 0:00:14
----- 29.4/56.8 MB 2.0 MB/s eta 0:00:14
----- 29.6/56.8 MB 2.0 MB/s eta 0:00:14
----- 30.1/56.8 MB 2.0 MB/s eta 0:00:14
----- 30.7/56.8 MB 2.0 MB/s eta 0:00:14
----- 30.9/56.8 MB 2.0 MB/s eta 0:00:13
----- 31.5/56.8 MB 2.0 MB/s eta 0:00:13
----- 32.2/56.8 MB 2.0 MB/s eta 0:00:13
----- 32.5/56.8 MB 2.0 MB/s eta 0:00:12
----- 32.8/56.8 MB 2.0 MB/s eta 0:00:12
----- 33.3/56.8 MB 2.0 MB/s eta 0:00:12
----- 33.6/56.8 MB 2.0 MB/s eta 0:00:12
----- 33.8/56.8 MB 2.0 MB/s eta 0:00:12
----- 34.3/56.8 MB 2.0 MB/s eta 0:00:12
----- 34.6/56.8 MB 2.0 MB/s eta 0:00:12

```

```

----- 34.9/56.8 MB 2.0 MB/s eta 0:00:12
----- 35.4/56.8 MB 2.0 MB/s eta 0:00:11
----- 35.7/56.8 MB 2.0 MB/s eta 0:00:11
----- 36.2/56.8 MB 2.0 MB/s eta 0:00:11
----- 36.7/56.8 MB 2.0 MB/s eta 0:00:11
----- 37.2/56.8 MB 2.0 MB/s eta 0:00:10
----- 37.5/56.8 MB 2.0 MB/s eta 0:00:10
----- 37.7/56.8 MB 2.0 MB/s eta 0:00:10
----- 38.0/56.8 MB 2.0 MB/s eta 0:00:10
----- 38.8/56.8 MB 2.0 MB/s eta 0:00:10
----- 39.3/56.8 MB 2.0 MB/s eta 0:00:09
----- 39.6/56.8 MB 2.0 MB/s eta 0:00:09
----- 40.4/56.8 MB 2.0 MB/s eta 0:00:09
----- 40.9/56.8 MB 2.0 MB/s eta 0:00:08
----- 41.4/56.8 MB 2.0 MB/s eta 0:00:08
----- 41.7/56.8 MB 2.0 MB/s eta 0:00:08
----- 41.9/56.8 MB 2.0 MB/s eta 0:00:08
----- 42.5/56.8 MB 2.0 MB/s eta 0:00:08
----- 43.0/56.8 MB 2.0 MB/s eta 0:00:07
----- 43.3/56.8 MB 2.0 MB/s eta 0:00:07
----- 43.5/56.8 MB 2.0 MB/s eta 0:00:07
----- 43.8/56.8 MB 2.0 MB/s eta 0:00:07
----- 44.0/56.8 MB 2.0 MB/s eta 0:00:07
----- 44.3/56.8 MB 2.0 MB/s eta 0:00:07
----- 44.8/56.8 MB 2.0 MB/s eta 0:00:07
----- 45.4/56.8 MB 2.0 MB/s eta 0:00:06
----- 45.4/56.8 MB 2.0 MB/s eta 0:00:06
----- 45.9/56.8 MB 2.0 MB/s eta 0:00:06
----- 46.4/56.8 MB 2.0 MB/s eta 0:00:06
----- 46.9/56.8 MB 2.0 MB/s eta 0:00:06
----- 47.2/56.8 MB 2.0 MB/s eta 0:00:05
----- 47.4/56.8 MB 2.0 MB/s eta 0:00:05
----- 48.0/56.8 MB 2.0 MB/s eta 0:00:05
----- 48.0/56.8 MB 2.0 MB/s eta 0:00:05
----- 48.5/56.8 MB 2.0 MB/s eta 0:00:05
----- 48.8/56.8 MB 2.0 MB/s eta 0:00:05
----- 49.3/56.8 MB 2.0 MB/s eta 0:00:04
----- 49.5/56.8 MB 2.0 MB/s eta 0:00:04
----- 49.8/56.8 MB 1.9 MB/s eta 0:00:04
----- 50.1/56.8 MB 1.9 MB/s eta 0:00:04
----- 50.6/56.8 MB 1.9 MB/s eta 0:00:04
----- 51.1/56.8 MB 1.9 MB/s eta 0:00:03
----- 51.4/56.8 MB 1.9 MB/s eta 0:00:03
----- 51.9/56.8 MB 1.9 MB/s eta 0:00:03
----- 52.4/56.8 MB 1.9 MB/s eta 0:00:03
----- 52.7/56.8 MB 1.9 MB/s eta 0:00:03
----- 53.0/56.8 MB 1.9 MB/s eta 0:00:03
----- 53.2/56.8 MB 1.9 MB/s eta 0:00:02

```

```

----- -- 53.7/56.8 MB 1.9 MB/s eta 0:00:02
----- - 54.0/56.8 MB 1.9 MB/s eta 0:00:02
----- - 54.3/56.8 MB 1.9 MB/s eta 0:00:02
----- - 54.8/56.8 MB 1.9 MB/s eta 0:00:02
----- - 55.1/56.8 MB 1.9 MB/s eta 0:00:01
----- - 55.3/56.8 MB 1.9 MB/s eta 0:00:01
----- 55.8/56.8 MB 1.9 MB/s eta 0:00:01
----- 56.4/56.8 MB 1.9 MB/s eta 0:00:01
----- 56.6/56.8 MB 1.9 MB/s eta 0:00:01
----- 56.8/56.8 MB 1.9 MB/s eta 0:00:00

```

Installing collected packages: xgboost
 Successfully installed xgboost-3.0.5

```

[13]: # Remove special characters from feature names
X_train_clean = X_train.copy()
X_test_clean = X_test.copy()
X_train_clean.columns = [c.replace('[', '').replace(']', '').replace('<', '').
    ↪replace(' ', '_') for c in X_train_clean.columns]
X_test_clean.columns = [c.replace('[', '').replace(']', '').replace('<', '').
    ↪replace(' ', '_') for c in X_test_clean.columns]

# Train XGBoost
import xgboost as xgb

xgb_model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss')
xgb_model.fit(X_train_clean, y_train)
y_pred_xgb = xgb_model.predict(X_test_clean)

from sklearn.metrics import classification_report
print("XGBoost:\n", classification_report(y_test, y_pred_xgb))

```

C:\Users\Priya\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarning:
 [22:02:12] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:738:
 Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

XGBoost:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	1932
1	0.83	0.66	0.74	68
accuracy			0.98	2000
macro avg	0.91	0.83	0.86	2000
weighted avg	0.98	0.98	0.98	2000

```
[15]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

nn_model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

nn_model.compile(optimizer=Adam(learning_rate=0.001),
                 loss='binary_crossentropy', metrics=['accuracy'])

history = nn_model.fit(X_train_scaled, y_train, validation_split=0.2,
                       epochs=50, batch_size=32)

# Evaluate
nn_eval = nn_model.evaluate(X_test_scaled, y_test)
print(f"Neural Network Test Loss: {nn_eval[0]:.4f}, Accuracy: {nn_eval[1]*100:.2f}%")
```

C:\Users\Priya\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:93:
 UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
 using Sequential models, prefer using an `Input(shape)` object as the first
 layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Epoch 1/50

```
200/200          5s 7ms/step -
accuracy: 0.9341 - loss: 0.3026 - val_accuracy: 0.9656 - val_loss: 0.1409
```

Epoch 2/50

```
200/200          1s 5ms/step -
accuracy: 0.9648 - loss: 0.1369 - val_accuracy: 0.9706 - val_loss: 0.1103
```

Epoch 3/50

```
200/200          1s 6ms/step -
accuracy: 0.9667 - loss: 0.1056 - val_accuracy: 0.9700 - val_loss: 0.1013
```

Epoch 4/50

```
200/200          1s 6ms/step -
accuracy: 0.9705 - loss: 0.0965 - val_accuracy: 0.9694 - val_loss: 0.0938
```

Epoch 5/50

```
200/200          1s 6ms/step -
accuracy: 0.9683 - loss: 0.0898 - val_accuracy: 0.9675 - val_loss: 0.0915
```

Epoch 6/50

```
200/200          1s 5ms/step -
accuracy: 0.9752 - loss: 0.0759 - val_accuracy: 0.9706 - val_loss: 0.0861
```

Epoch 7/50

```
200/200          1s 5ms/step -
```

accuracy: 0.9717 - loss: 0.0845 - val_accuracy: 0.9725 - val_loss: 0.0790
 Epoch 8/50
 200/200 1s 6ms/step -
 accuracy: 0.9709 - loss: 0.0839 - val_accuracy: 0.9706 - val_loss: 0.0799
 Epoch 9/50
 200/200 1s 5ms/step -
 accuracy: 0.9760 - loss: 0.0740 - val_accuracy: 0.9719 - val_loss: 0.0747
 Epoch 10/50
 200/200 1s 5ms/step -
 accuracy: 0.9760 - loss: 0.0711 - val_accuracy: 0.9706 - val_loss: 0.0734
 Epoch 11/50
 200/200 1s 6ms/step -
 accuracy: 0.9769 - loss: 0.0675 - val_accuracy: 0.9694 - val_loss: 0.0741
 Epoch 12/50
 200/200 1s 6ms/step -
 accuracy: 0.9794 - loss: 0.0612 - val_accuracy: 0.9706 - val_loss: 0.0698
 Epoch 13/50
 200/200 1s 6ms/step -
 accuracy: 0.9767 - loss: 0.0663 - val_accuracy: 0.9762 - val_loss: 0.0668
 Epoch 14/50
 200/200 1s 5ms/step -
 accuracy: 0.9757 - loss: 0.0709 - val_accuracy: 0.9719 - val_loss: 0.0688
 Epoch 15/50
 200/200 1s 5ms/step -
 accuracy: 0.9777 - loss: 0.0619 - val_accuracy: 0.9781 - val_loss: 0.0623
 Epoch 16/50
 200/200 1s 5ms/step -
 accuracy: 0.9765 - loss: 0.0634 - val_accuracy: 0.9787 - val_loss: 0.0613
 Epoch 17/50
 200/200 1s 6ms/step -
 accuracy: 0.9819 - loss: 0.0524 - val_accuracy: 0.9719 - val_loss: 0.0655
 Epoch 18/50
 200/200 1s 5ms/step -
 accuracy: 0.9802 - loss: 0.0568 - val_accuracy: 0.9719 - val_loss: 0.0615
 Epoch 19/50
 200/200 1s 5ms/step -
 accuracy: 0.9821 - loss: 0.0514 - val_accuracy: 0.9737 - val_loss: 0.0588
 Epoch 20/50
 200/200 1s 5ms/step -
 accuracy: 0.9800 - loss: 0.0610 - val_accuracy: 0.9756 - val_loss: 0.0601
 Epoch 21/50
 200/200 1s 5ms/step -
 accuracy: 0.9813 - loss: 0.0626 - val_accuracy: 0.9756 - val_loss: 0.0575
 Epoch 22/50
 200/200 1s 5ms/step -
 accuracy: 0.9801 - loss: 0.0576 - val_accuracy: 0.9781 - val_loss: 0.0571
 Epoch 23/50
 200/200 1s 5ms/step -

accuracy: 0.9820 - loss: 0.0489 - val_accuracy: 0.9725 - val_loss: 0.0640
 Epoch 24/50
 200/200 1s 5ms/step -
 accuracy: 0.9805 - loss: 0.0544 - val_accuracy: 0.9762 - val_loss: 0.0575
 Epoch 25/50
 200/200 1s 5ms/step -
 accuracy: 0.9816 - loss: 0.0506 - val_accuracy: 0.9781 - val_loss: 0.0584
 Epoch 26/50
 200/200 1s 5ms/step -
 accuracy: 0.9817 - loss: 0.0510 - val_accuracy: 0.9756 - val_loss: 0.0625
 Epoch 27/50
 200/200 1s 6ms/step -
 accuracy: 0.9789 - loss: 0.0541 - val_accuracy: 0.9775 - val_loss: 0.0569
 Epoch 28/50
 200/200 1s 5ms/step -
 accuracy: 0.9799 - loss: 0.0535 - val_accuracy: 0.9769 - val_loss: 0.0578
 Epoch 29/50
 200/200 1s 5ms/step -
 accuracy: 0.9810 - loss: 0.0536 - val_accuracy: 0.9775 - val_loss: 0.0557
 Epoch 30/50
 200/200 1s 6ms/step -
 accuracy: 0.9823 - loss: 0.0541 - val_accuracy: 0.9775 - val_loss: 0.0561
 Epoch 31/50
 200/200 1s 5ms/step -
 accuracy: 0.9829 - loss: 0.0578 - val_accuracy: 0.9762 - val_loss: 0.0580
 Epoch 32/50
 200/200 1s 4ms/step -
 accuracy: 0.9839 - loss: 0.0495 - val_accuracy: 0.9775 - val_loss: 0.0552
 Epoch 33/50
 200/200 1s 5ms/step -
 accuracy: 0.9806 - loss: 0.0541 - val_accuracy: 0.9794 - val_loss: 0.0525
 Epoch 34/50
 200/200 1s 5ms/step -
 accuracy: 0.9817 - loss: 0.0559 - val_accuracy: 0.9794 - val_loss: 0.0538
 Epoch 35/50
 200/200 1s 5ms/step -
 accuracy: 0.9811 - loss: 0.0539 - val_accuracy: 0.9787 - val_loss: 0.0541
 Epoch 36/50
 200/200 1s 5ms/step -
 accuracy: 0.9797 - loss: 0.0574 - val_accuracy: 0.9787 - val_loss: 0.0536
 Epoch 37/50
 200/200 1s 5ms/step -
 accuracy: 0.9829 - loss: 0.0515 - val_accuracy: 0.9719 - val_loss: 0.0669
 Epoch 38/50
 200/200 1s 5ms/step -
 accuracy: 0.9834 - loss: 0.0482 - val_accuracy: 0.9775 - val_loss: 0.0549
 Epoch 39/50
 200/200 1s 4ms/step -


```

accuracy: 0.9843 - loss: 0.0483 - val_accuracy: 0.9794 - val_loss: 0.0519
Epoch 40/50
200/200          1s 5ms/step -
accuracy: 0.9833 - loss: 0.0504 - val_accuracy: 0.9781 - val_loss: 0.0548
Epoch 41/50
200/200          1s 5ms/step -
accuracy: 0.9815 - loss: 0.0487 - val_accuracy: 0.9769 - val_loss: 0.0568
Epoch 42/50
200/200          1s 5ms/step -
accuracy: 0.9825 - loss: 0.0477 - val_accuracy: 0.9775 - val_loss: 0.0574
Epoch 43/50
200/200          1s 5ms/step -
accuracy: 0.9823 - loss: 0.0491 - val_accuracy: 0.9762 - val_loss: 0.0542
Epoch 44/50
200/200          1s 5ms/step -
accuracy: 0.9845 - loss: 0.0487 - val_accuracy: 0.9762 - val_loss: 0.0541
Epoch 45/50
200/200          1s 5ms/step -
accuracy: 0.9832 - loss: 0.0498 - val_accuracy: 0.9794 - val_loss: 0.0546
Epoch 46/50
200/200          1s 5ms/step -
accuracy: 0.9858 - loss: 0.0435 - val_accuracy: 0.9787 - val_loss: 0.0519
Epoch 47/50
200/200          1s 5ms/step -
accuracy: 0.9846 - loss: 0.0484 - val_accuracy: 0.9812 - val_loss: 0.0529
Epoch 48/50
200/200          1s 5ms/step -
accuracy: 0.9826 - loss: 0.0504 - val_accuracy: 0.9775 - val_loss: 0.0541
Epoch 49/50
200/200          1s 5ms/step -
accuracy: 0.9783 - loss: 0.0598 - val_accuracy: 0.9750 - val_loss: 0.0569
Epoch 50/50
200/200          1s 5ms/step -
accuracy: 0.9844 - loss: 0.0420 - val_accuracy: 0.9750 - val_loss: 0.0641
63/63           1s 4ms/step -
accuracy: 0.9746 - loss: 0.0681
Neural Network Test Loss: 0.0654, Accuracy: 97.55%

```

```

[17]: import numpy as np
      from tensorflow.keras.layers import LSTM

      # Example: 10-minute sequences
      sequence_length = 10
      X_seq, y_seq = [], []

      for i in range(len(X_train_scaled) - sequence_length):
          X_seq.append(X_train_scaled[i:i+sequence_length])

```

```

        y_seq.append(y_train.iloc[i+sequence_length])

X_seq = np.array(X_seq)
y_seq = np.array(y_seq)

lstm_model = Sequential([
    LSTM(64, input_shape=(sequence_length, X_train_scaled.shape[1])),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

lstm_model.compile(optimizer=Adam(0.001), loss='binary_crossentropy',
    metrics=['accuracy'])
history_lstm = lstm_model.fit(X_seq, y_seq, epochs=50, batch_size=32)

```

Epoch 1/50

C:\Users\Priya\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:199:
 UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
 using Sequential models, prefer using an `Input(shape)` object as the first
 layer in the model instead.

```
super().__init__(**kwargs)
```

250/250 7s 12ms/step -

accuracy: 0.9069 - loss: 0.3131

Epoch 2/50

250/250 3s 13ms/step -

accuracy: 0.9632 - loss: 0.1580

Epoch 3/50

250/250 3s 12ms/step -

accuracy: 0.9674 - loss: 0.1444

Epoch 4/50

250/250 3s 12ms/step -

accuracy: 0.9673 - loss: 0.1440

Epoch 5/50

250/250 3s 13ms/step -

accuracy: 0.9653 - loss: 0.1500

Epoch 6/50

250/250 3s 13ms/step -

accuracy: 0.9648 - loss: 0.1526

Epoch 7/50

250/250 3s 12ms/step -

accuracy: 0.9645 - loss: 0.1525

Epoch 8/50

250/250 3s 13ms/step -

accuracy: 0.9650 - loss: 0.1511

Epoch 9/50

250/250 3s 14ms/step -

```

accuracy: 0.9666 - loss: 0.1458
Epoch 10/50
250/250          3s 13ms/step -
accuracy: 0.9655 - loss: 0.1482
Epoch 11/50
250/250          3s 13ms/step -
accuracy: 0.9694 - loss: 0.1367
Epoch 12/50
250/250          3s 13ms/step -
accuracy: 0.9685 - loss: 0.1374
Epoch 13/50
250/250          3s 13ms/step -
accuracy: 0.9644 - loss: 0.1514
Epoch 14/50
250/250          3s 13ms/step -
accuracy: 0.9640 - loss: 0.1531
Epoch 15/50
250/250          3s 10ms/step -
accuracy: 0.9681 - loss: 0.1390
Epoch 16/50
250/250          3s 11ms/step -
accuracy: 0.9651 - loss: 0.1474
Epoch 17/50
250/250          3s 11ms/step -
accuracy: 0.9648 - loss: 0.1465
Epoch 18/50
250/250          3s 13ms/step -
accuracy: 0.9638 - loss: 0.1499
Epoch 19/50
250/250          3s 11ms/step -
accuracy: 0.9663 - loss: 0.1432
Epoch 20/50
250/250          3s 13ms/step -
accuracy: 0.9669 - loss: 0.1409
Epoch 21/50
250/250          3s 10ms/step -
accuracy: 0.9713 - loss: 0.1231
Epoch 22/50
250/250          3s 13ms/step -
accuracy: 0.9673 - loss: 0.1377
Epoch 23/50
250/250          3s 13ms/step -
accuracy: 0.9648 - loss: 0.1474
Epoch 24/50
250/250          3s 13ms/step -
accuracy: 0.9664 - loss: 0.1343
Epoch 25/50
250/250          3s 14ms/step -

```

```

accuracy: 0.9644 - loss: 0.1426
Epoch 26/50
250/250          4s 14ms/step -
accuracy: 0.9663 - loss: 0.1328
Epoch 27/50
250/250          3s 11ms/step -
accuracy: 0.9681 - loss: 0.1278
Epoch 28/50
250/250          3s 10ms/step -
accuracy: 0.9684 - loss: 0.1240
Epoch 29/50
250/250          3s 10ms/step -
accuracy: 0.9685 - loss: 0.1203
Epoch 30/50
250/250          3s 11ms/step -
accuracy: 0.9693 - loss: 0.1151
Epoch 31/50
250/250          5s 10ms/step -
accuracy: 0.9658 - loss: 0.1205
Epoch 32/50
250/250          3s 10ms/step -
accuracy: 0.9704 - loss: 0.1104
Epoch 33/50
250/250          3s 12ms/step -
accuracy: 0.9674 - loss: 0.1113
Epoch 34/50
250/250          3s 13ms/step -
accuracy: 0.9713 - loss: 0.1000
Epoch 35/50
250/250          3s 13ms/step -
accuracy: 0.9718 - loss: 0.0930
Epoch 36/50
250/250          3s 13ms/step -
accuracy: 0.9685 - loss: 0.0931
Epoch 37/50
250/250          3s 13ms/step -
accuracy: 0.9763 - loss: 0.0742
Epoch 38/50
250/250          3s 11ms/step -
accuracy: 0.9750 - loss: 0.0721
Epoch 39/50
250/250          3s 12ms/step -
accuracy: 0.9759 - loss: 0.0681
Epoch 40/50
250/250          3s 11ms/step -
accuracy: 0.9788 - loss: 0.0609
Epoch 41/50
250/250          3s 11ms/step -

```

```

accuracy: 0.9813 - loss: 0.0559
Epoch 42/50
250/250          6s 14ms/step -
accuracy: 0.9826 - loss: 0.0501
Epoch 43/50
250/250          3s 12ms/step -
accuracy: 0.9869 - loss: 0.0401
Epoch 44/50
250/250          3s 10ms/step -
accuracy: 0.9883 - loss: 0.0342
Epoch 45/50
250/250          3s 11ms/step -
accuracy: 0.9910 - loss: 0.0276
Epoch 46/50
250/250          3s 11ms/step -
accuracy: 0.9882 - loss: 0.0327
Epoch 47/50
250/250          3s 11ms/step -
accuracy: 0.9948 - loss: 0.0195
Epoch 48/50
250/250          3s 11ms/step -
accuracy: 0.9960 - loss: 0.0153
Epoch 49/50
250/250          3s 10ms/step -
accuracy: 0.9957 - loss: 0.0145
Epoch 50/50
250/250          3s 10ms/step -
accuracy: 0.9941 - loss: 0.0153

```

```

[19]: # Assuming sequence_length = 10
X_test_seq, y_test_seq = [], []

for i in range(len(X_test_scaled) - sequence_length):
    X_test_seq.append(X_test_scaled[i:i+sequence_length])
    y_test_seq.append(y_test.iloc[i+sequence_length])

X_test_seq = np.array(X_test_seq)
y_test_seq = np.array(y_test_seq)

```

```

[21]: # Evaluate test loss and accuracy
test_loss, test_accuracy = lstm_model.evaluate(X_test_seq, y_test_seq)
print(f"LSTM Test Loss: {test_loss:.4f}")
print(f"LSTM Test Accuracy: {test_accuracy*100:.2f}%")

```

```

63/63          1s 8ms/step -
accuracy: 0.9383 - loss: 0.5544
LSTM Test Loss: 0.4905
LSTM Test Accuracy: 94.37%

```

```
[23]: from sklearn.metrics import classification_report

# Predict probabilities and convert to class labels
y_pred_probs = lstm_model.predict(X_test_seq)
y_pred = (y_pred_probs > 0.5).astype(int)

print("LSTM Classification Report:\n", classification_report(y_test_seq,
    ↪y_pred))
```

```
63/63          1s 14ms/step
LSTM Classification Report:
```

	precision	recall	f1-score	support
0	0.97	0.98	0.97	1922
1	0.02	0.01	0.02	68
accuracy			0.94	1990
macro avg	0.49	0.50	0.49	1990
weighted avg	0.93	0.94	0.94	1990

```
[25]: # Random Forest predictions
from sklearn.metrics import accuracy_score, precision_score, recall_score,
    ↪f1_score, classification_report
import numpy as np

y_pred_rf = model.predict(X_test)
rf_metrics = {
    'Accuracy': accuracy_score(y_test, y_pred_rf),
    'Precision': precision_score(y_test, y_pred_rf),
    'Recall': recall_score(y_test, y_pred_rf),
    'F1-score': f1_score(y_test, y_pred_rf)
}

# XGBoost predictions
y_pred_xgb = xgb_model.predict(X_test_clean)
xgb_metrics = {
    'Accuracy': accuracy_score(y_test, y_pred_xgb),
    'Precision': precision_score(y_test, y_pred_xgb),
    'Recall': recall_score(y_test, y_pred_xgb),
    'F1-score': f1_score(y_test, y_pred_xgb)
}
```

```
C:\Users\Priya\anaconda3\Lib\site-packages\sklearn\base.py:486: UserWarning: X
has feature names, but RandomForestClassifier was fitted without feature names
warnings.warn(
```

```
[27]: # Predict probabilities and convert to class labels
y_pred_nn_probs = nn_model.predict(X_test_scaled)
y_pred_nn = (y_pred_nn_probs > 0.5).astype(int)

nn_metrics = {
    'Accuracy': accuracy_score(y_test, y_pred_nn),
    'Precision': precision_score(y_test, y_pred_nn),
    'Recall': recall_score(y_test, y_pred_nn),
    'F1-score': f1_score(y_test, y_pred_nn)
}
```

63/63 0s 5ms/step

```
[29]: # LSTM test sequences prepared as before
y_pred_lstm_probs = lstm_model.predict(X_test_seq)
y_pred_lstm = (y_pred_lstm_probs > 0.5).astype(int)

lstm_metrics = {
    'Accuracy': accuracy_score(y_test_seq, y_pred_lstm),
    'Precision': precision_score(y_test_seq, y_pred_lstm),
    'Recall': recall_score(y_test_seq, y_pred_lstm),
    'F1-score': f1_score(y_test_seq, y_pred_lstm)
}
```

63/63 0s 7ms/step

```
[31]: import pandas as pd

comparison_df = pd.DataFrame({
    'Random Forest': rf_metrics,
    'XGBoost': xgb_metrics,
    'MLP': nn_metrics,
    'LSTM': lstm_metrics
}).T

comparison_df = comparison_df.round(3)
comparison_df
```

```
[31]:
```

	Accuracy	Precision	Recall	F1-score
Random Forest	0.034	0.034	1.000	0.066
XGBoost	0.984	0.833	0.662	0.738
MLP	0.976	0.880	0.324	0.473
LSTM	0.944	0.022	0.015	0.018

```
[33]: joblib.dump(xgb_model, "xgb_model_ai4i.joblib")
joblib.dump(scaler, "scaler_ai4i.joblib")
print(" Model and Scaler saved successfully!")
```

Model and Scaler saved successfully!

```
[55]: import shap

# Load model and scaler
xgb_model = joblib.load("xgb_model_ai4i.joblib")
scaler = joblib.load("scaler_ai4i.joblib")

FEATURES = ['Air temperature [K]', 'Process temperature [K]',
            'Rotational speed [rpm]', 'Torque [Nm]',
            'Tool wear [min]']

# Initialize SHAP TreeExplainer (use unscaled input)
explainer = shap.TreeExplainer(xgb_model)
```

```
[1]: import pandas as pd
import joblib

xgb_model = joblib.load("xgb_model_ai4i.joblib")
scaler = joblib.load("scaler_ai4i.joblib")

df1 = pd.DataFrame([[310,320,1500,50,50]],
                    columns=['Air_temperature', 'Process_temperature', 'Rotational_speed', 'Torque', 'Tool_wear'])
df2 = pd.DataFrame([[330,300,1200,60,20]], columns=df1.columns)

print(xgb_model.predict(df1), xgb_model.predict(df2))
print(xgb_model.predict_proba(df1), xgb_model.predict_proba(df2))
```

```
[0] [1]
[[9.9995786e-01 4.2128890e-05]] [[0.07933539 0.9206646 ]]
```

```
[5]: # Import libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
import xgboost as xgb
import shap

# -----
# Step 1: Load Dataset
# -----
data = pd.read_csv('ai4i2020.csv') # Replace with your path
```



```

# -----
# Step 2: Identify & Encode Categorical Columns
# -----
# Find object type columns
cat_cols = data.select_dtypes(include=['object']).columns

# Encode categorical columns using LabelEncoder
le = LabelEncoder()
for col in cat_cols:
    data[col] = le.fit_transform(data[col])

# -----
# Step 3: Split Features and Target
# -----
X = data.drop(columns=['Machine_failure'])
y = data['Machine_failure']

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Step 4: Standardize Features
# -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# Step 5: Train XGBoost Model
# -----
xg_model = xgb.XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
xg_model.fit(X_train_scaled, y_train)

# Get predicted probabilities for Logistic Regression
xg_probs = xg_model.predict_proba(X_train_scaled)[: , 1].reshape(-1, 1)
xg_test_probs = xg_model.predict_proba(X_test_scaled)[: , 1].reshape(-1, 1)

# -----
# Step 6: Train Logistic Regression on XGBoost output
# -----
log_model = LogisticRegression()

```

```

log_model.fit(xg_probs, y_train)

# Final Predictions
final_preds = log_model.predict(xg_test_probs)

# -----
# Step 7: Evaluate Model
# -----
print("Accuracy:", accuracy_score(y_test, final_preds))
print("Confusion Matrix:\n", confusion_matrix(y_test, final_preds))
print("Classification Report:\n", classification_report(y_test, final_preds))

# -----
# Step 8: SHAP Explainability
# -----
explainer = shap.KernelExplainer(log_model.predict_proba, xg_probs)
shap_values = explainer.shap_values(xg_test_probs)

# Summary Plot
shap.summary_plot(shap_values, X_test_scaled)

```

C:\Users\Priya\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarning:
[19:24:03] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

bst.update(dtrain, iteration=i, fobj=obj)
Using 8000 background data samples could cause slower run times. Consider using
shap.sample(data, K) or shap.kmeans(data, K) to summarize the background as K
samples.

Accuracy: 0.999
Confusion Matrix:

```
[[1939   0]
 [   2  59]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1939
1	1.00	0.97	0.98	61
accuracy			1.00	2000
macro avg	1.00	0.98	0.99	2000
weighted avg	1.00	1.00	1.00	2000

0%| | 0/2000 [00:00<?, ?it/s]

```

AssertionError                                Traceback (most recent call last)
Cell In[5], line 82
    79 shap_values = explainer.shap_values(xg_test_probs)
    81 # Summary Plot
--> 82 shap.summary_plot(shap_values, X_test_scaled)

File ~\anaconda3\Lib\site-packages\shap\plots\_beeswarm.py:664, in summary_legacy(shap_values, features, feature_names, max_display, plot_type, color, axis_color, title, alpha, show, sort, color_bar, plot_size, layered_violin_max_num_bins, class_names, class_inds, color_bar_label, cmap, show_values_in_legend, use_log_scale, rng)
    659         raise ValueError(
    660             shape_msg + " Perhaps the extra column in the shap_values matrix is the "
    661             "constant offset? Of so just pass shap_values[:, :-1].")
    662     )
    663     else:
--> 664         assert num_features == features.shape[1], shape_msg
    666 if feature_names is None:
    667     feature_names = np.array([labels["FEATURE"] % str(i) for i in range(num_features)])

AssertionError: The shape of the shap_values matrix does not match the shape of
the provided data matrix.

```

```

[9]: # Import libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import xgboost as xgb
import shap
import joblib

# -----
# Step 1: Load Dataset
# -----
data = pd.read_csv('ai4i2020.csv') # Replace with your local path

# -----
# Step 2: Encode Categorical Columns
# -----
cat_cols = data.select_dtypes(include=['object']).columns
le = LabelEncoder()

```

```

for col in cat_cols:
    data[col] = le.fit_transform(data[col])

# -----
# Step 3: Split Features and Target
# -----
X = data.drop(columns=['Machine_failure'])
y = data['Machine_failure']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Step 4: Standardize Features
# -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# Step 5: Train XGBoost Model
# -----
xg_model = xgb.XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
xg_model.fit(X_train_scaled, y_train)

# XGBoost predicted probabilities (for Logistic Regression)
xg_train_probs = xg_model.predict_proba(X_train_scaled)[: , 1].reshape(-1, 1)
xg_test_probs = xg_model.predict_proba(X_test_scaled)[: , 1].reshape(-1, 1)

# -----
# Step 6: Train Logistic Regression on XGBoost output
# -----
log_model = LogisticRegression()
log_model.fit(xg_train_probs, y_train)

# Final predictions
final_preds = log_model.predict(xg_test_probs)

# -----
# Step 7: Evaluate Model
# -----
print("Accuracy:", accuracy_score(y_test, final_preds))

```

```

print("Confusion Matrix:\n", confusion_matrix(y_test, final_preds))
print("Classification Report:\n", classification_report(y_test, final_preds))

# -----
# Step 8: SHAP Explainability for XGBoost
# -----
explainer = shap.TreeExplainer(xg_model)
shap_values = explainer.shap_values(X_test_scaled)

# Summary plot
shap.summary_plot(shap_values, X_test_scaled, feature_names=X.columns)

# -----
# Step 9: Save Models and Scaler
# -----
joblib.dump(xg_model, 'xgboost_model.pkl')
joblib.dump(log_model, 'logistic_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
print("Models and scaler saved successfully!")

```

C:\Users\Priya\anaconda3\Lib\site-packages\xgboost\training.py:183: UserWarning:
[19:28:53] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

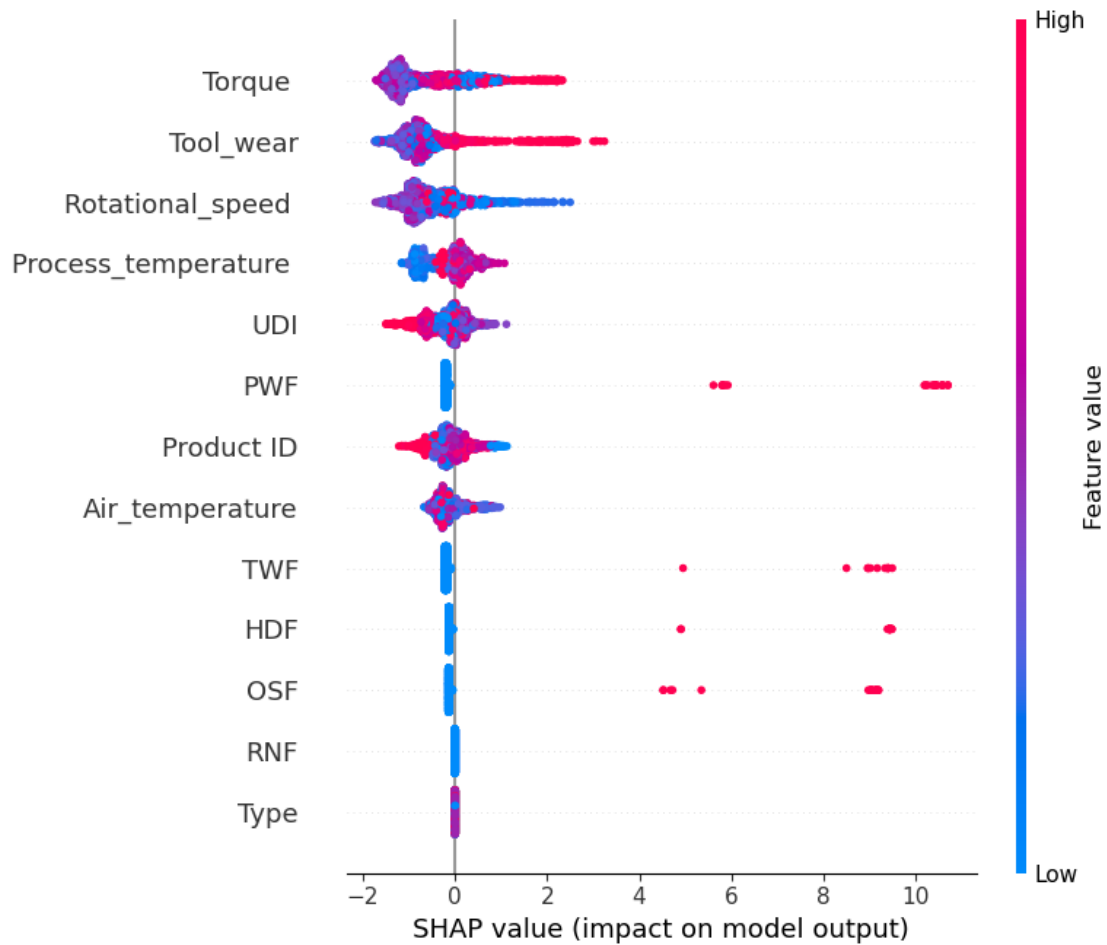
Accuracy: 0.999

Confusion Matrix:

```
[[1939   0]
 [  2  59]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1939
1	1.00	0.97	0.98	61
accuracy			1.00	2000
macro avg	1.00	0.98	0.99	2000
weighted avg	1.00	1.00	1.00	2000



Models and scaler saved successfully!

```
[11]: # Save models explicitly as .joblib files
joblib.dump(xg_model, 'xgboost_model.joblib')
joblib.dump(log_model, 'logistic_model.joblib')
joblib.dump(scaler, 'scaler.joblib')

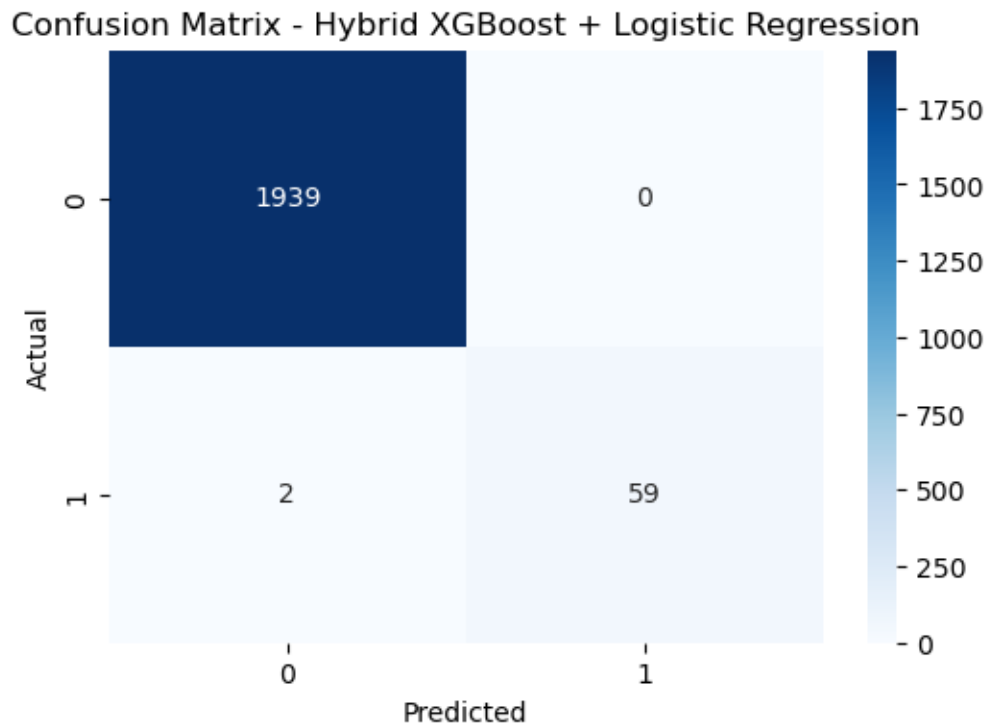
print("Models and scaler saved successfully in joblib format!")
```

Models and scaler saved successfully in joblib format!

```
[13]: # Import libraries for visualization
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Compute confusion matrix
cm = confusion_matrix(y_test, final_preds)
```

```
# Visualize using seaborn heatmap
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=[0,1],
            yticklabels=[0,1])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Hybrid XGBoost + Logistic Regression')
plt.show()
```



[]: